Lucid App Updates: component design

Oct 3, 2026 · @Sushil

Overview

Six components deliver signed, region-correct APK updates to vehicles and report back on how each update went. This document covers them one at a time, starting with the API and Manifest.

Scope for Phase 1:

- APK-based apps only, updated in place. No new apps can be added through the updater.
- The vehicle runs Android without Google Play. A privileged system app installs updates silently.
- The design is cloud-agnostic. Regions are US, EU and KSA, resolved by the cloud from the VIN. Not every app is offered in every region: Spotify is not offered in KSA and SiriusXM is not offered in EU.
- Auto-update runs only when the vehicle is on WiFi. Manual update is always allowed.
- Full APKs only. A rollback ships the previous version as a new, higher versionCode.
- Staged rollout (5, 10, 20, 50, 100 percent) and auto-halt are phase 2.

Component	What it does
DevOps	Builds, versions and signs APKs from the existing CI
Release Manager	Owns release records, approvals, QA debug to Release prod promotion and publishing
API and Manifest	Identifies the vehicle, resolves its region and returns a signed manifest
Storage and CDN	Holds APKs per region and serves downloads
Analytics	Collects update events and derives release health
Vehicle updater	Checks, downloads, verifies and installs updates under the auto and manual policy

1. API and Manifest

The API and Manifest service is the only place a vehicle goes to learn which updates it may install. It identifies the vehicle, resolves its region and returns a signed list of available updates. When the vehicle is ready to download, it returns the download details for the apps the vehicle picks.

Responsibilities

- Authenticate the vehicle by its device certificate (mTLS) and take the VIN from the certificate.
- Resolve the vehicle's region from the VIN, so the vehicle never holds a region setting.
- Build the manifest: for each app the vehicle reports as installed, the newest published version available in its region.
- Return download details on request: for each package and version the vehicle asks for, a size, a hash and a short-lived download URL on the regional CDN.
- Sign the manifest so the vehicle can verify it came from the cloud.
- Accept published and withdrawn releases from the Release Manager.

Not responsible for

- Building, signing or approving APKs. That belongs to DevOps and the Release Manager.
- Storing or serving APK files. That belongs to Storage and CDN.
- Deciding when to install. The vehicle updater applies the WiFi and manual rules.
- Collecting update events. The vehicle sends those to Analytics.
- Offering new apps. The manifest answers only for apps the vehicle already has.

Interfaces

The service exposes two vehicle-facing calls and two internal calls. Endpoint names are proposals.

Interface	Caller	What it does	Auth
POST /v1/manifest	Vehicle updater	Sends installed apps and versions. Returns a signed manifest listing the updates available	Device certificate (mTLS)
POST /v1/manifestapks	Vehicle updater	Sends the packages and versions it wants to download. Returns a size, SHA-256 and short-lived download URL for each	Device certificate (mTLS)
POST /internal/v1/releases	Release Manager	Publishes a release: app, version, regions, APK location, hash, size	Service identity
POST /internal/v1/releases/{id}/withdraw	Release Manager	Stops offering a release. Used for rollback	Service identity
Download URL from /v1/manifestapks	Vehicle to CDN	Fetches one APK	Short-lived token in the URL

/v1/manifest takes the installed packages with their versionCode, plus the OS version. /v1/manifestapks takes the packages the vehicle wants, each with its versionName and versionCode. Neither call carries the VIN, which comes from the device certificate. /v1/manifestapks answers only for versions published in the vehicle's region.

Data it owns

The service holds a published copy of release data. The Release Manager stays the source of truth for versions.

- **Published releases:** app, versionName, versionCode, regions where it is available, APK location, SHA-256, size, status (published or withdrawn).

- **Region lookup:** the VIN-to-region rule. A VIN does not reliably show the market a vehicle was sold in, so this should come from vehicle master data (see open questions).
- **Trust configuration:** the certificate authority that issues device certificates, and the revocation list.
- **Signing key reference:** a pointer to the manifest signing key in KMS or HSM. The service never holds the private key.

Manifest schema

The manifest lists only updates for apps the vehicle reported. An empty list means nothing to install. Field names are proposals.

```
{
  "schemaVersion": 1,
  "generatedAt": "2026-10-03T09:15:00Z",
  "expiresAt": "2026-10-04T09:15:00Z",
  "region": "EU",
  "updates": [
    {
      "releaseId": "rel-4.1.0",
      "package": "com.example.media",
      "versionName": "4.1.0",
      "versionCode": 40100
    }
  ],
  "signature": { "alg": "ES256", "keyId": "manifest-key-1", "value":
"MEUC..." }
}
```

- **versionCode** only ever goes up, because Android blocks downgrades. A rollback is the previous build re-released with a higher versionCode.
- **expiresAt** is short (24 hours), so an old manifest cannot be replayed later.
- **No download details here.** Size, hash and URL come from /v1/manifestapks, so the manifest stays small and URLs are fresh when the vehicle uses them.
- **releaseId** is shared by every update in the same release.
- **signature** covers the whole body. The updater checks it before trusting any field.

Download details: POST /v1/manifestapks

The vehicle sends the packages it wants to install. The server returns one entry per package, keyed by package and versionCode. The vehicle calls it when the download starts, and again if a URL expires.

Request:

```
{
  "apps": [
    { "package": "com.example.media", "versionName": "4.1.0", "versionCode":
40100 }
  ]
}
```

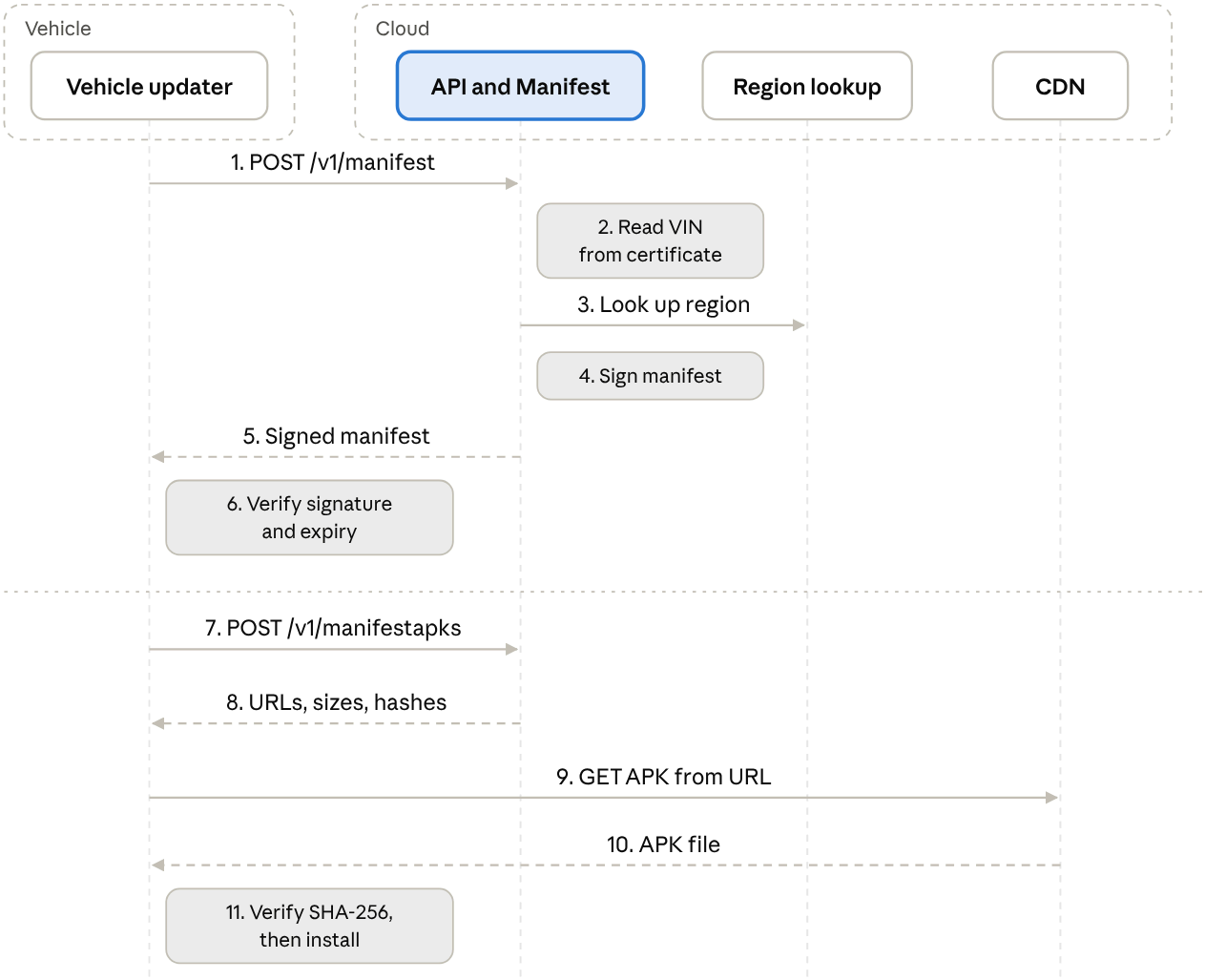
Response:

```
{
  "apps": [
    {
      "package": "com.example.media",
      "versionCode": 40100,
      "sizeBytes": 48211968,
      "sha256": "9f2c...e1",
      "url": "https://cdn-eu.updates.example/media/40100/media-4.1.0.apk?
token=...",
      "urlExpiresAt": "2026-10-03T10:15:00Z"
    }
  ]
}
```

How it works

The vehicle first asks what is available. It asks for download details only when it is ready to install.

Updates are listed first; download URLs are requested only at install



manifest and download flow · 11 steps, 4 participants

Steps 2 to 4 happen in the cloud: the certificate gives the VIN, the VIN gives the region, and the API signs the manifest with its signing key (see Security). In steps 6 and 11 the vehicle checks the signature and the APK hash. Dashed lines are replies.

Security

Four checks stand between the cloud and an installed APK, and each one covers a different attack.

- **Device identity:** each vehicle holds its own certificate. The API requires mTLS and reads the VIN from the certificate, so a vehicle cannot ask for another VIN's manifest or URLs. Revoked certificates are rejected.

- **Signed manifest:** signed with a key held in a key service (KMS or HSM, explained below). The updater carries the public key in the system image, plus a second key so keys can rotate.
- **APK integrity:** the SHA-256 for each APK arrives with the download details, and the updater checks it after download. Android also verifies the APK signature, and the signing certificate must match the installed app.
- **Freshness:** the updater rejects an expired manifest, and download URLs are short-lived and valid for one APK in one region.

On the internal side, only the Release Manager's service identity can publish or withdraw a release, and every publish is logged.

Failure modes

The rule throughout: when something fails, the vehicle keeps running the apps it has and the cloud never guesses.

Failure	What happens
API unreachable	The updater keeps the current apps and retries with backoff. Manual check works once the connection returns.
Region cannot be resolved	The API returns an error and no updates. It never falls back to a default region, because of data residency. An alert is raised.
Manifest signature invalid or expired	The updater discards it, records an event and retries later.
Download URL expired	The updater calls /v1/manifestapks again for fresh URLs. No new manifest is needed.
Release withdrawn	The next manifest no longer lists it. An install already in progress finishes.
Signing key unavailable	No new manifests are issued. Vehicles keep their apps and receive no updates until the key is back.

Phase 1 and Phase 2

In Phase 1 every vehicle in a region sees a published release at once. Phase 2 adds control over who sees it, and when.

Area	Phase 1	Phase 2
Who gets a release	All eligible vehicles in the region	Percentage ramp of 5, 10, 20, 50 and 100 percent within a region
Rollout decision	None. Published means available	A Rollout Service decides per vehicle. A hash of the VIN keeps each vehicle in the same bucket
Halting	Withdraw the release by hand	Auto-halt when Analytics reports install failures above a threshold
Download size	Full APKs	Delta patches

Open questions

Each question has a default, which this section assumes until you decide otherwise.

- [ ] **Region source:** where does the market a vehicle belongs to come from? Default: a lookup of the VIN in vehicle master data, not a VIN prefix rule.
- [ ] **Check frequency:** when does the updater ask for a manifest? Default: at startup and every 24 hours while connected, plus manual check.
- [ ] **Signing key ownership:** who holds the manifest signing key and runs rotation? Default: KMS or HSM with two active keys.
- [ ] **Compatibility:** does any app need a minimum OS version in v1? Default: yes, a minimum OS version only.

2. Release Manager

The Release Manager sits between a built APK and a vehicle. It decides which builds are released, in what order, and when the cloud starts offering them. It is a backend service with a web UI.

Responsibilities

- Receive signed debug and prod APKs and their build details from DevOps, and create a release record for each.
- Own the version registry: every versionName and versionCode ever released for each app. Reject a build whose versionCode is not higher than the latest one.
- Run the approval flow: a release moves through QA debug, QA prod, Release debug and Release prod, and each step needs an approval by a named person.
- Publish an approved release: copy the APK to Storage and CDN in each region where the app is offered, then publish the release to API and Manifest. Only prod builds are published to vehicles.
- Withdraw a release, and manage rollback by re-releasing the previous build under a higher versionCode.
- Show release health from Analytics: installs, failures and versions in use, per release and region.
- Keep an audit trail of who approved, promoted, published or withdrew what, and when.
- Phase 2: control the staged rollout (5, 10, 20, 50, 100 percent) and act on auto-halt.

Not responsible for

- Building or signing APKs. That belongs to DevOps.
- Answering vehicle requests or serving APK files. That belongs to API and Manifest and to Storage and CDN.
- Installing anything on a vehicle. That belongs to the Vehicle updater.
- Collecting update events. Analytics does that, and the Release Manager only reads the results.

Release lifecycle

Stage	Meaning	Moves on when
Build	Signed debug and prod builds and details arrived from DevOps and passed the intake checks	Automatically
QA debug	Debug build under QA test	The QA approver approves
QA prod	Production build under QA test	The QA approver approves

Stage	Meaning	Moves on when
Release debug	Debug build in release approval	The release approver approves
Release prod	Production build approved and published to the regions where the app is offered	A release manager publishes
Withdrawn	No longer offered. Vehicles that already have it keep it	A release manager withdraws it

Versioning

The rules below are proposals. DevOps stamps the version at build time and the Release Manager enforces the rules at intake.

- versionName is major.minor.patch, for example 4.1.0. Versioning starts at 4.0.0.
- versionCode is derived from versionName: major x 10000 + minor x 100 + patch, so 4.1.0 gives 40100. It only goes up when the version goes up.
- The registry rejects a build with a duplicate or lower versionCode.
- A rollback is the previous build re-released as a new patch version with a higher versionCode, then the bad release is withdrawn.
- A release has its own version and lists the app versions it contains. For example, Release 2 is 4.2.0 and contains Media 4.2.0 and Navigation 4.1.0.

Interfaces

Interface	Who	What it does
Web UI	Release managers and approvers	Review builds, approve, promote, publish, withdraw and see release health
Release intake	DevOps	Accepts the signed debug and prod APKs with their build details and accepts or rejects it
POST /internal/v1/releases	Release Manager calls API and Manifest	Publishes a release

Interface	Who	What it does
POST /internal/v1/releases/{id}/withdraw	Release Manager calls API and Manifest	Stops offering a release
APK upload	Release Manager to Storage and CDN	Copies the APK to each target region
Release health	Release Manager reads from Analytics	Install results per release and region

Data it owns

- **Release records:** release name and version, then for each app in the release: versionName, versionCode, and for its debug and prod builds the build id, SHA-256, size and signing certificate fingerprint. Also target regions, stage and timestamps.
- **Version registry:** the source of truth for every released version of every app.
- **Approvals and audit log:** who did what, and when.

It does not own the APK files (Storage and CDN) or the published copy of releases (API and Manifest).

Failure modes

Failure	What happens
Build has a duplicate or lower versionCode	Rejected at intake with the reason. DevOps fixes the version and resubmits.
Publish fails in one region	The APK is copied to every region before the release is published, so a manifest never lists a missing file. If any region fails, nothing is published and the failed region is shown.
Approver unavailable	The release waits. Nothing is approved automatically.
Bad release found	Withdraw it and publish the rollback release.
Release Manager is down	Vehicles are not affected, because they never call it. New releases pause until it is back.

Phase 1 and Phase 2

Area	Phase 1	Phase 2
Visibility	Release prod means every eligible vehicle in the region can see it	Percentage ramp within a region
Halting	A release manager withdraws by hand	Auto-halt when Analytics reports failures above a threshold
Health	Dashboard of installs and failures	Thresholds and alerts

Open questions

- [ ] **QA and Release stages:** are they approval gates only, or do test vehicles receive the build? Default: gates only. Vehicles see a release only at Release prod.
- [ ] **Approvals:** how many approvers per stage, and is there a compliance sign-off before Release prod? Default: one approver per stage and no external gate.
- [ ] **versionCode rule:** is deriving it from versionName acceptable? Default: yes.
- [ ] **Rollback naming:** default is a new patch version, for example 4.1.1.
- [ ] **App availability by region:** where is the list of regions per app kept? Default: the Release Manager keeps it and publishes an app only to the regions where it is offered.

3. Vehicle and Vehicle updater

The vehicle is the platform. The Vehicle updater is the app on it that does all the update work. They share a section because the updater depends on what the vehicle gives it.

What the vehicle provides

- The updater built into the system image as a privileged, platform-signed app.
- Signals the updater reads: whether the car is parked, whether it is on WiFi, and free storage.
- The apps themselves, each signed by its own certificate.

Vehicle updater responsibilities

- Check for updates by calling /v1/manifest with the installed apps and versions. Default: at startup and every 24 hours while connected, plus a manual check.

- Verify the manifest signature and expiry before using it.
- Apply the update policy: auto-update only when on WiFi, and manual update always allowed.
- Call /v1/manifestapks for the apps it will install, then download the APKs and resume if interrupted. Apps in the same release are downloaded and installed together.
- Check each APK's SHA-256 against the download details. Android verifies the APK signature at install.
- Install silently through the PackageInstaller API, with no prompts.
- Move each update through the lifecycle below and show its status in the updater UI.
- Send an event to Analytics at each lifecycle step, and queue events while offline.
- Never install an app that is not already on the vehicle, and never install a lower versionCode.

Not responsible for

- Deciding which versions exist or are available. The cloud decides.
- Resolving the vehicle's region. The cloud does that from the VIN.
- Hosting or serving APKs.
- Updating the OS, firmware or ECUs. That is a separate pipeline outside this scope.
- Judging whether a release is healthy. It only reports events.

Update lifecycle

State	Meaning	Moves on to
Idle	Nothing pending	Update available, when a manifest lists a newer version
Update available	A newer version is listed	Waiting, or Downloading
Waiting	Auto-update is on but the car is not on WiFi	Downloading, once conditions are met or the driver taps Update now
Downloading	The APK is being fetched. It resumes if interrupted	Verifying
Verifying	Hash is checked	Installing, or Failed

State	Meaning	Moves on to
Installing	Silent install in progress	Installed, or Failed
Installed	The new version is running	Idle
Failed	A step failed. The old version stays in place	Update available again, on a later retry or when the driver taps Retry

Auto and manual updates

	Auto-update	Manual update
Starts when	The car is on WiFi	The driver taps Update now
Driver prompt	None	None beyond the tap
If conditions change	Pauses if WiFi drops, resumes later	Not affected by WiFi
Setting	Driver can turn auto-update off	Always available

Interfaces

Interface	Direction	What it does
POST /v1/manifest	Updater to API and Manifest	Gets the list of available updates
POST /v1/manifestapks	Updater to API and Manifest	Gets size, hash and a short-lived URL for each app to download
APK download	Updater to CDN	Fetches one APK
Lifecycle events	Updater to Analytics	Reports each step of an update
PackageInstaller	Updater to Android	Installs an APK silently
Updater UI	Driver to updater	Shows status, and offers Update now and Retry

Data it keeps on the vehicle

- The installed apps and their versions, read from Android.

- The last valid manifest and its expiry.
- Each pending update with its lifecycle state and retry count.
- Downloaded APKs, kept only until installed and then deleted.
- A queue of events not yet sent.
- The driver's auto-update setting.

Security

- The updater is a platform-signed system app, which is what lets it install without prompts.
- It checks the manifest signature and expiry, the APK hash, and refuses any downgrade.
- It refuses to install a package that is not already installed.
- Downloaded APKs sit in a private directory that other apps cannot read.

Failure modes

Failure	What happens
Connection drops during download	The download pauses and resumes later. The partial file is kept.
Not enough storage	The update does not start. The status says so and an event is sent.
Hash does not match	The file is deleted and the update fails. A later retry gets fresh URLs.
Install fails	The old version stays in place. The update is marked Failed and an event is sent.
Vehicle restarts mid-update	The updater reads its saved state on startup and resumes or cleans up.
Cloud unreachable	The updater keeps the current apps and retries with backoff.

Phase 2

The vehicle side changes little in Phase 2. Rollout decisions stay in the cloud, because the manifest either lists an update for a vehicle or does not. The one vehicle change is delta patches: the updater downloads only the difference and rebuilds the APK before the hash check.

Open questions

- [ ] **Install while driving:** manual updates are always allowed, but should the install step wait until the car is parked?
- [ ] **Manual update on cellular:** allowed? Default: yes, with the download size shown first.
- [ ] **Retry policy:** default is three attempts with backoff, then wait for the next check or a manual retry.
- [ ] **Safety-relevant apps:** are any too sensitive to auto-update? Default: none in Phase 1.

4. DevOps

DevOps turns source code into signed, correctly versioned APKs and hands them to the Release Manager. It uses the existing CI, so engineers keep the build process they know.

Responsibilities

- Let development build the debug and prod APKs for each app in a release from the existing CI, on demand or from a tag.
- Stamp versionName and versionCode using the versioning rules in the Release Manager section.
- Run the build checks: tests, lint, and a check that the package is one of the apps already on vehicles, because new apps are out of scope.
- Sign prod builds with the release key and debug builds with the debug key. The keys live in a key service, so the CI never holds the private key.
- Submit the signed debug and prod APKs to the Release Manager with its build details: package, versionName, versionCode, SHA-256, signing certificate fingerprint, minimum OS version, build id and commit.
- Keep build logs and artifacts so any release can be traced back to its source.

Not responsible for

- Approving, promoting or publishing a release. That belongs to the Release Manager.
- Deciding which vehicles or regions get a release.
- Storing APKs for vehicles. That belongs to Storage and CDN.
- Tracking how an update went on vehicles. That belongs to Analytics.

Interfaces

Interface	Direction	What it does
Source repository	Development to CI	A tag or manual trigger starts a release build
Key service	CI to key service	Signs the APK without exposing the private key
Release intake	CI to Release Manager	Submits the signed debug and prod APKs and build details
Intake result	Release Manager to CI	Accepted, or rejected with a reason such as a lower versionCode

Data it owns

- Build artifacts, logs and build details.
- The record linking each APK to its commit.

The signing keys are not stored in DevOps. They sit in the key service.

Security

- Release builds run only from protected branches or tags.
- Each app has its own signing key. The signing certificate must stay the same across versions, because Android refuses an update signed by a different certificate. A debug build uses a separate debug key, so it cannot replace a production app.
- Only the CI can request a signature, and every signing request is logged.

Failure modes

Failure	What happens
Build or tests fail	Nothing is submitted. The engineer sees the failure.
Key service unavailable	The build waits, and no unsigned APK is ever submitted.
Release Manager rejects the version	The CI shows the reason. The engineer fixes the version and rebuilds.
Package is not an existing app	The build fails the update-only check.

Open questions

- [ ] **CI system:** which one is in use today? Default: the existing CI, with a release job added.
- [ ] **Signing key ownership:** who owns the keys and the key service? Default: the security team, with a release key for prod builds and a debug key for debug builds.
- [ ] **Build trigger:** can any engineer start a release build? Default: tags on protected branches only.

5. Analytics

Analytics collects update events from vehicles and turns them into release health that the Release Manager can act on. It answers one question: did this release install cleanly?

Responsibilities

- Receive events from vehicles, authenticated by the device certificate.
- Validate and store them, and drop duplicates, because a vehicle may resend events after being offline.
- Derive metrics per release and region: manifest checks, downloads, installs that succeeded or failed, failure reasons, time to install, and the share of vehicles on each version.
- Provide release health to the Release Manager.
- Phase 2: signal when failures cross a threshold, so the rollout can halt automatically.
- Apply the retention rules to stored events.

Not responsible for

- Deciding to halt or roll back a release. The Release Manager decides, and in Phase 2 a Rollout Service acts on the signal.
- Collecting driver, location or app-usage data.
- Storing releases, manifests or APKs.

Events

The vehicle sends one event per lifecycle step. Names and fields are proposals.

Event	Sent when	Extra fields
manifest_checked	A manifest check finishes	Number of updates listed, or the error
update_waiting	Auto-update cannot start	Reason: not on WiFi
download_started	A download begins	Size
download_completed	A download finishes	Duration
install_started	An install begins	None
install_succeeded	The new version is running	Duration
install_failed	An update fails	Reason: hash mismatch, low storage, install error or other

Every event also carries: event id (for deduplication), release id, VIN(Hashed), region, package, from versionCode, to versionCode, trigger (auto or manual), updater version and timestamp.

Interfaces

Interface	Direction	What it does
Event ingestion	Vehicle updater to Analytics	Accepts events in batches over mTLS

Interface	Direction	What it does
Release health	Analytics to Release Manager	Returns install results per release and region
Threshold signal	Analytics to Rollout Service	Phase 2 only. Reports that failures crossed a threshold

Data it owns

- Raw events.
- Aggregates per release, region and version.
- Release health results.

Privacy

Events describe updates only. They carry no driver identity, location or app-usage data. The VIN is needed to count vehicles and deduplicate, so how it is stored and for how long is an open question below.

Failure modes

Failure	What happens
Vehicle offline	The updater queues events and sends them later. Event time, not arrival time, is used in the metrics.
Ingestion unavailable	The updater keeps its queue and retries.
Duplicate events	Dropped by event id.
No events yet for a release	Health shows no data, never healthy.

Phase 1 and Phase 2

Area	Phase 1	Phase 2
Output	Counts and failure rates for people to read	Thresholds, alerts and an automatic halt signal

Area	Phase 1	Phase 2
Decision	A release manager reads the dashboard and withdraws by hand	The Rollout Service halts the ramp automatically

Open questions

- [ ] **Privacy:** are there GDPR or consent requirements for VIN-keyed events in the EU or KSA? Default: update events only, VIN stored with region-specific retention.
- [ ] **Retention:** how long are raw events kept? Default: 12 months raw, with aggregates kept longer.
- [ ] **At-risk level:** when does a release show as at risk? Default: failures above 2 percent of attempted installs, once at least 50 installs have been attempted. These numbers are placeholders.